

avito.tech ⚡

ПОРОЖДАЮЩИЕ ПАТТЕРНЫ

Афанасьев Юра

Выносят из клиентского кода
привязки к конкретным классам,
абстрагируя процесс
инстанцирования

Решаемые задачи

- Скрывают детали создания (конструкцию `new class()`) в абстракции
- Инкапсулируют знания о конкретных классах
- Удовлетворяют принципу слабая связанность (low coupling) из GRASP

1. **Abstract Factory.** Абстрактная фабрика

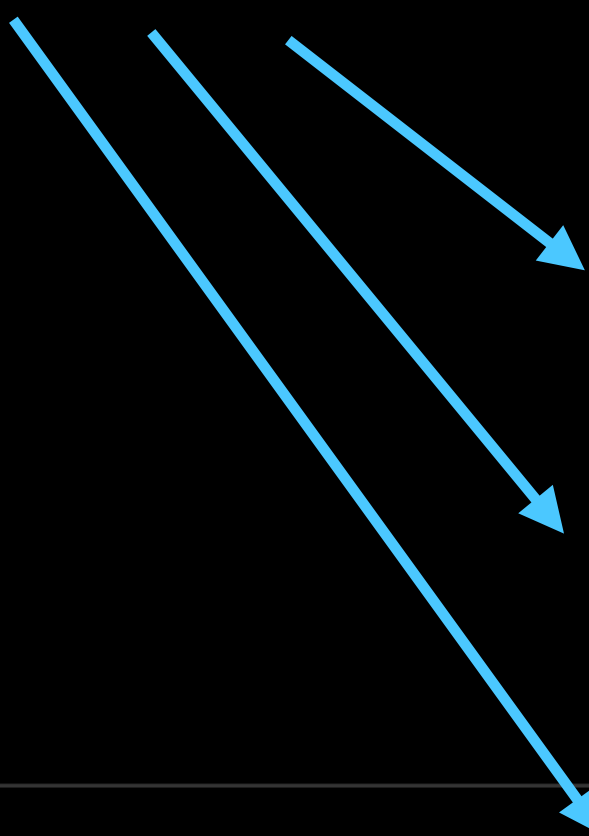
2. **Builder.** Строитель

3. **Factory Method.** Фабричный метод

4. **Prototype.** Прототип

5. **Singleton.** Одиночка

Фабрикой называют что-то
среднее между этими паттернами



1. **Abstract Factory.** Абстрактная фабрика

2. **Builder.** Строитель

3. **Factory Method.** Фабричный метод

4. **Prototype.** Прототип

5. **Singleton.** Одиночка

BUILDER



СТРОИТЕЛЬ

Отделяет конструирование
сложного объекта от его
представления, так что в результате
одного и того же процесса
конструирования могут получаться
разные представления

Механика

Выносит логику для создания целевого класса в специальный класс, который будет накапливать данные и по готовности создавать объект

```
class Order
{
    public function getBasketItems() {}
    public function removeItemFromBasket() {}
    public function deliveryGoods() {}
    public function storeInWarehouse() {}
    public function sendNotification() {}
    /* и так далее... */
}
```


Зависимости разбросаны
по нескольким классам

```
class Order
{
    public function __construct(
        private IBasket $basket,
        private IDelivery $delivery,
        private IWarehouse $warehouse,
        private IMailer $mailer
    ) {
    }
}
```

Слишком много
аргументов

Решаемая задача

- Требуется растянуть во времени процесс создания объекта
- Значения для аргументов конструктора целевого класса разбросаны по нескольким классам
- Нужно сократить количество аргументов конструктора до одного

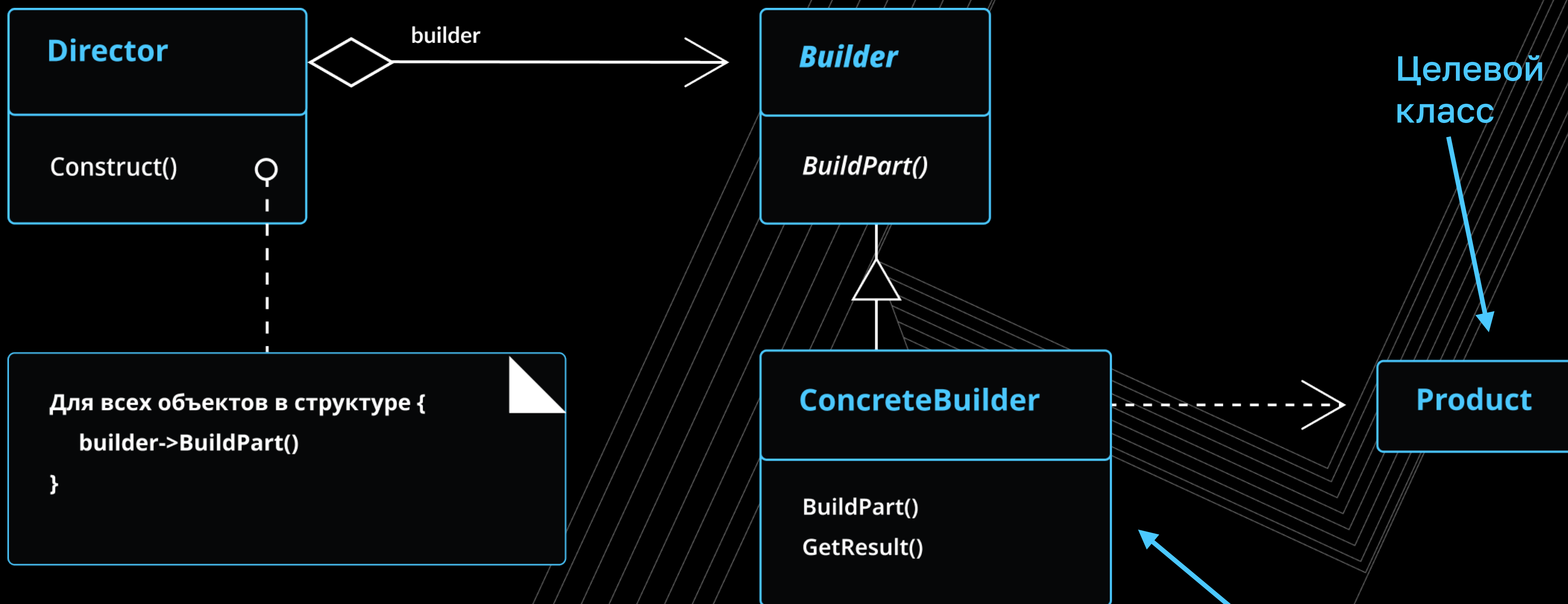
Распорядитель
(знает последовательность
шагов для создания
класса)

Чаще всего
не используются

Интерфейс
строителя

Целевой
класс

Строитель



Целевой
класс

Строитель

```
class Product
{
    private string $property;
    // private $anotherProperty;

    public function __construct(ConcreteBuilder $builder)
    {
        $this->property = $builder->getProperty();
        // $this->anotherProperty = $builder->getAnotherProperty();
    }
}
```

Каждому свойству соответствует
свой метод строителя

Строитель

Каждому свойству
создаём свой
сеттер и геттер

Строитель создаёт
целевой класс

В качестве аргумента
передаём самого себя

```
class ConcreteBuilder
{
    private string $property;
    // private $anotherProperty;

    public function setProperty(string $property): ConcreteBuilder
    {
        $this->property = $property;
        return $this;
    }

    public function getProperty(): string
    {
        return $this->property;
    }

    // public function setAnotherProperty(mixed $property): ConcreteBuilder {}
    // public function getAnotherProperty(): mixed { }

    public function build(): Product
    {
        return new Product($this);
    }
}
```

Количество свойств
равно количеству
свойств целевого класса

Пробрасываем
зависимость

```
function testBuilder()
{

    $builder = (new ConcreteBuilder())
               ->setProperty('test');

    $product = $builder->build();

    // some asserts
}
```

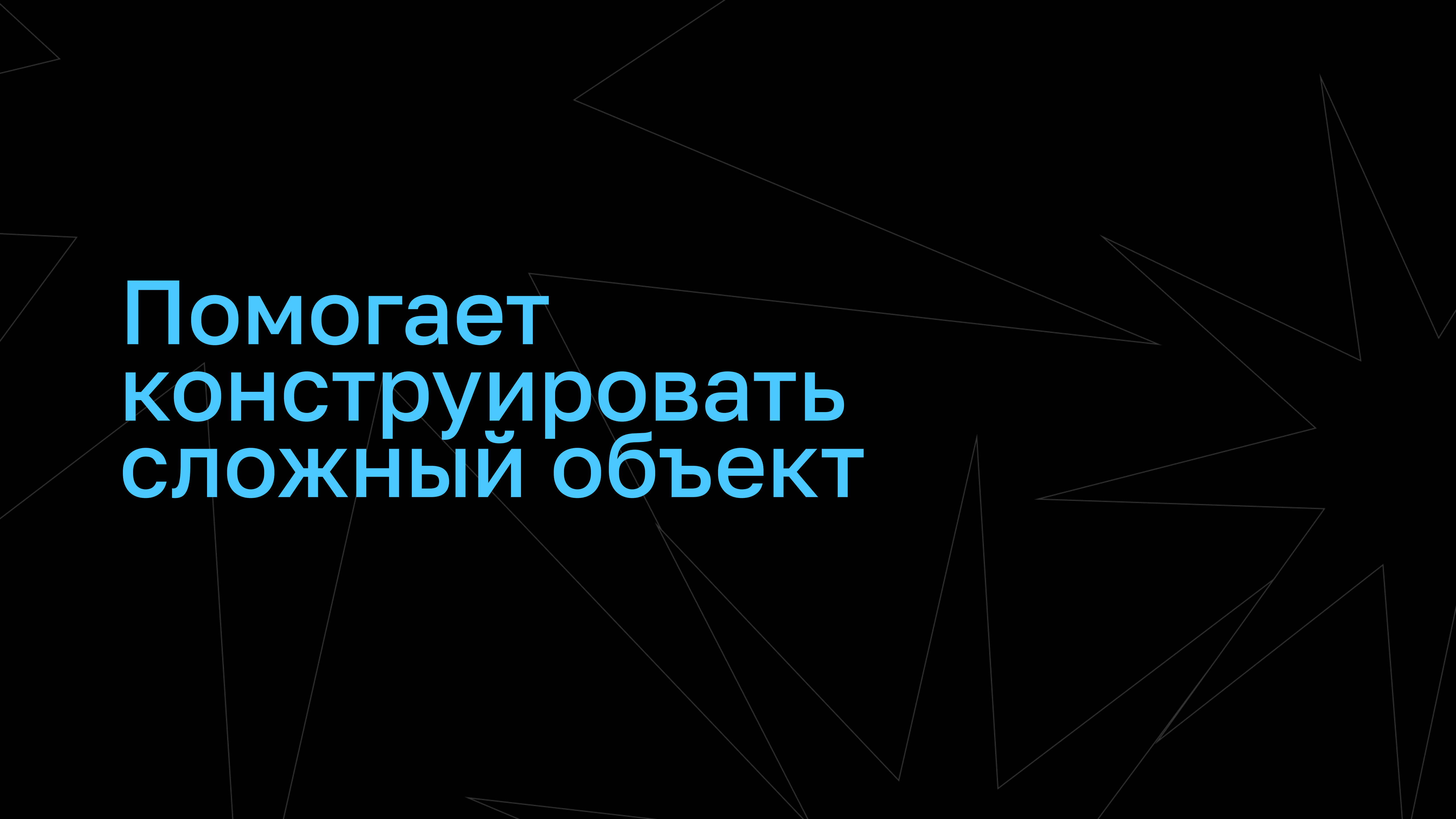
Как только Строитель
будет собран,
можно создавать объект

Плюсы

- Упрощает процесс создания сложного объекта
- Растягивает во времени процесс создания объекта
- Разделяет на два класса основной код и данные для конструирования
- Удовлетворяет принципу открытости/закрытости из SOLID и слабой связанности (low coupling) из GRASP

Минусы

- Добавляется новая сущность, которую также нужно поддерживать
- Клиент отвязывается от целевого класса, но создаёт связь со Строителем



Помогает
конструировать
сложный объект

FACTORY METHOD



ФАБРИЧНЫЙ МЕТОД

Определяет интерфейс
для создания объекта,
но оставляет подклассам решение
о том, какой класс инстанцировать

Позволяет классу делегировать
инстанцирование подклассам

Механика

Перемещает все операции инстанцирования
в свои отдельные фабричные методы
и, уже вызывая их, производит
создание объектов

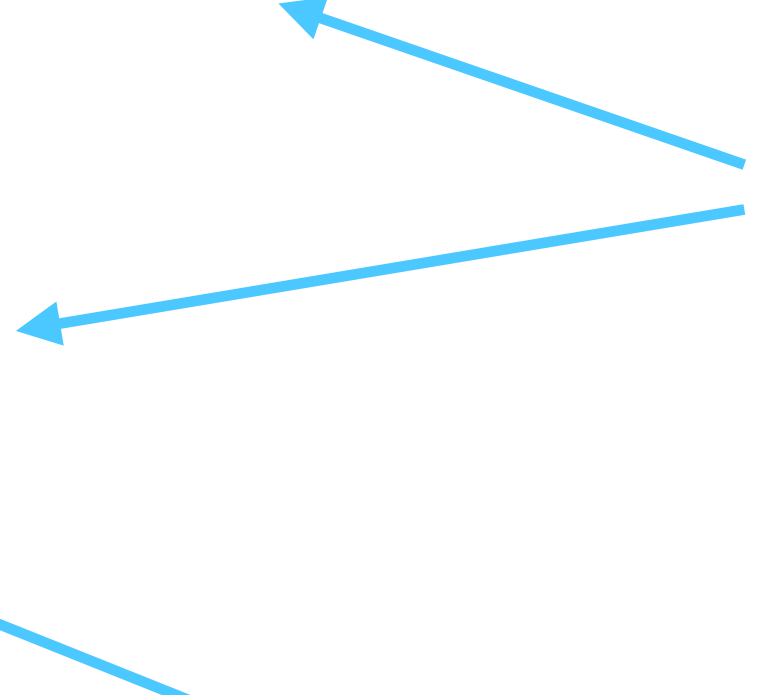

```
class Order
{
    public function getTotalAmount()
    {
        // ...КОД...

        $amount = $this->getAmount()->calculate();

        // ...КОД...
    }

    protected function getAmount()
    {
        return new Amount();
    }
}
```

Создание
класса
выделено
в метод



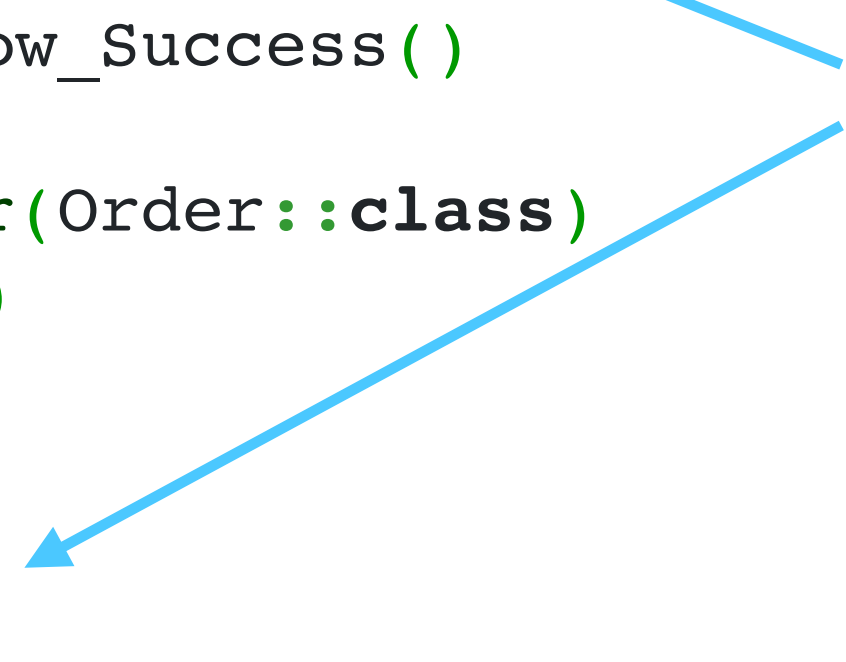
```
class OrderTest extends TestCase
{
    public function testGetTotalAmount_Flow_Success()
    {
        $orderMock = $this->getMockBuilder(Order::class)
            ->disableOriginalConstructor()
            ->getMock();

        $orderMock->method('getAmount')
            ->willReturn($amountMock);

        $amount = $orderMock->getTotalAmount();

        $this->assertEquals(100, $amount);
    }
}
```

Amount легко
замокать



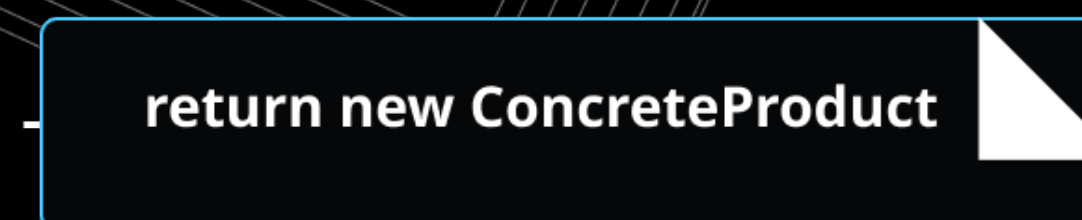
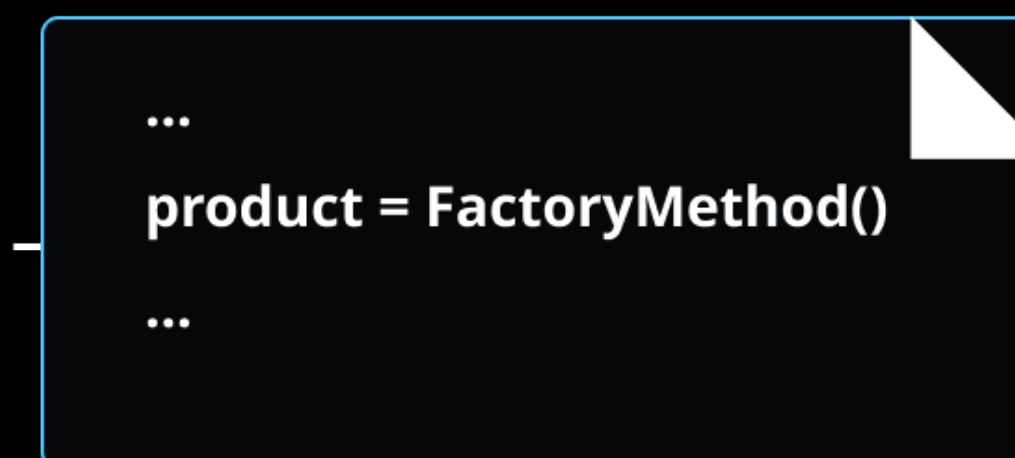
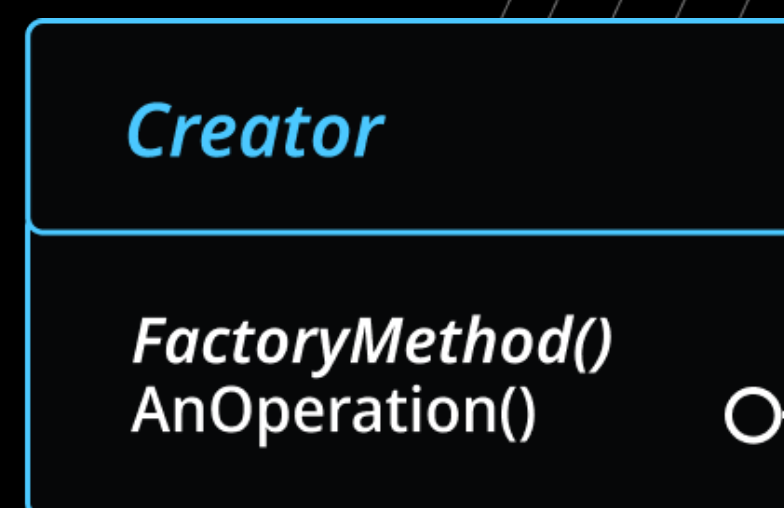
Решаемая задача

- Создаваемый класс необходимо изменить без переписывания кода
- Целевой класс работает с интерфейсом, а не конкретной реализацией
- Удобно для мока (mock) классов в юнит-тестах

Реализация
продукты



Создатель
(с фабричным методом)



Конкретный создатель
(переопределяет создателя)


```
abstract class Creator
{
    public function AnOperation()
    {
        // ...КОД...

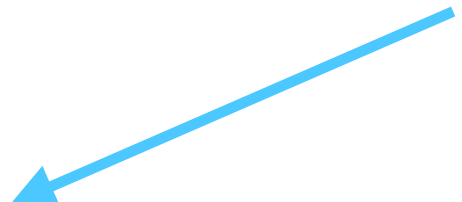
        $class = $this->factoryMethod();

        // ...КОД...
    }

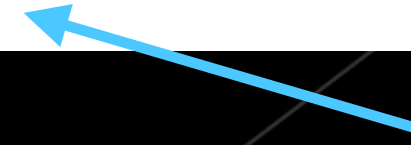
    abstract protected function factoryMethod();

    // abstract protected function anotherFactoryMethod();
}
```

Фабричный
метод



Фабричных методов
может быть много



На каждый
инвариант создаётся
наследник

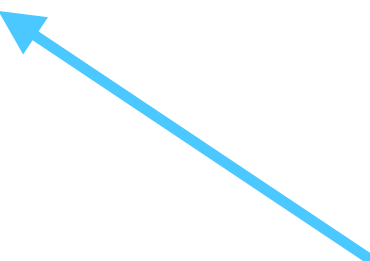
```
class ConcreteCreator extends Creator
{
    protected function factoryMethod()
    {
        return new ConcreteProduct();
    }

    // protected function anotherFactoryMethod() { }
}
```

```
class AnotherConcreteCreator extends Creator
{
    protected function factoryMethod()
    {
        return new AnotherConcreteProduct();
    }
}
```

Здесь можно
переопределить
Фабричный метод

```
function testFactoryMethod()  
{  
    $product = (new ConcreteCreator())->AnOperation();  
  
    // $product2 = (new AnotherConcreteCreator())->AnOperation();  
  
    // some asserts  
}
```



Используя разных
наследников,
меняем поток системы

Плюсы

- Избавляет классы от зависимости друг от друга
- Уменьшает связь с другими сущностями программы
- Фабричный метод более гибок, чем непосредственное создание объектов
- Улучшает тестируемость кода

Минусы

- Для изменения создаваемого класса необходимо каждый раз создавать новые подклассы
- Большое число фабричных методов ведёт к множеству наследников, для покрытия всех возможных вариантов

Особенности

- Фабричный метод может как иметь реализацию в конкретном классе, так и быть абстрактным
- Вместо наследования можно передавать аргумент в фабрику и по нему создавать конкретный класс

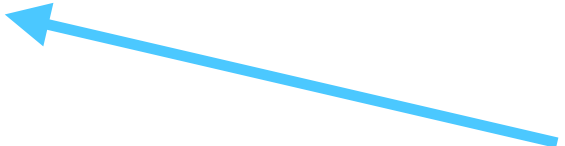

```
class Creator
{
    public function someOperation()
    {
        // ...КОД...

        $class = $this->factoryMethod();

        // ...КОД...
    }

    protected function factoryMethod()
    {
        return new ConcreteProduct();
    }
}
```

Дефолтная
реализация



```
class Creator
{
    public function someOperation()
    {
        // ...КОД...

        $class = $this->factoryMethod($className);

        // ...КОД...
    }

    protected function factoryMethod(string $className)
    {
        if ($className === 'ConcreteProduct') {
            return new ConcreteProduct();
        } elseif ($className === 'AnotherConcreteProduct') {
            return new AnotherConcreteProduct();
        }

        throw new LogicalException('Unknown class ' . $className);
    }
}
```

Метка для
создания объекта



Абстрагирует процесс создания классов, при этом выбор конкретного класса определяется подклассами

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

